

Corner Store Assistant

Practice exercise: an agent with tools, memory, streaming and tracing

1. Objective

Build an interactive terminal assistant for a small store. The agent must rely on tools for every fact (stock and prices) instead of guessing, remember what was said earlier in the conversation, print its answers while they are being generated, and leave a clear trail in the OpenAI traces dashboard. You will practice agent creation, system prompts, the `@function_tool` decorator, the Runner, conversation memory, streaming and tracing.

2. Store data

Keep the store data in a plain Python dictionary. No external API is needed. Prices are in AED.

Product	Price (AED)	Stock
milk	7.00	12
bread	4.50	0
eggs	12.00	8
apples	3.00	30
coffee	25.00	5

3. Functional requirements: tools

The agent has three tools. Each one must have a clear docstring, because the model reads it to decide when to call the tool.

Tool	Input	Behavior
check_stock	product name	Returns how many units are in stock. Reports clearly when the product does not exist in the store.
get_price	product name	Returns the unit price. Reports clearly when the product does not exist.
calculate_total	list of items, each with a product name and a quantity	Returns the total cost of the basket by multiplying each unit price by its quantity.

4. Business rules for the system prompt

Write a system prompt that makes the agent follow these rules. How you word them is up to you.

#	Rule
1	The agent is a friendly assistant for the store, with a short and polite tone.
2	It must never state a price or stock level from its own knowledge. Every number comes from a tool or from an earlier tool result in the conversation.
3	If a product does not exist, it says so politely and mentions what the store does sell.
4	If the customer asks for more units than are in stock, it tells them the available quantity and does not confirm the full order.
5	Questions unrelated to the store are declined politely, without calling any tool.
6	Follow-up messages such as "how much for 5 of them?" or "add 2 coffees to that" are understood from the earlier conversation. If the reference is truly unclear, the agent asks which product the customer means.

5. Memory requirements

The assistant runs as a chat loop in the terminal and keeps going until the user types exit. Every user message and every agent reply of the session must be stored, and on every turn the agent must receive the earlier messages as context so follow-up questions work. Choose one of the two options below. SQLite is the more ambitious choice because the conversation can survive a restart.

Option	Expected behavior
Local list of dicts	Each message is stored as a dictionary with at least a role and the content. The memory lives only while the script is running.
SQLite	Each message is saved in a table with a session id, role, content and timestamp. When the script starts, the user can type an existing session id and continue that conversation, even after a restart.

6. Streaming requirements

The agent's reply must appear in the terminal progressively, as it is generated, and not all at once at the end. When the agent starts using a tool, the terminal shows a short line with the tool name and its arguments, and another indication when the tool has finished. After the stream ends, the complete reply is saved to memory exactly like in the non-streaming version.

7. Tracing requirements

Every chat session must be visible in the OpenAI traces dashboard under a custom workflow name of your choice, for example Corner Store Assistant. Each turn produces its own trace, but all turns of the same session must be linked by a shared identifier so you can filter them together. Use the same session id for memory and for tracing, so a conversation stored in your memory can always be found in the dashboard.

8. Non-functional requirements

The script runs from the terminal. The API key is read from an environment variable and never written in the code. A failing tool or a network error must not crash the chat loop: the user sees a short error message and can keep chatting.

9. Example conversations

These examples show the behavior you should aim for. Tool calls and results are what you should see in the trace. The exact wording of the agent will differ from run to run, but the tool usage and the numbers should match.

Example A: chaining two tools

YOU	Do you have milk, and how much would 3 cost?
TOOL CALL	<code>check_stock(product="milk")</code>
TOOL RESULT	12 units in stock
TOOL CALL	<code>get_price(product="milk")</code>
TOOL RESULT	7.00 AED
TOOL CALL	<code>calculate_total(items=[{"product": "milk", "quantity": 3}])</code>
TOOL RESULT	21.00 AED
AGENT	Yes, we have milk in stock! It costs 7.00 AED each, so 3 bottles come to 21.00 AED. Anything else I can help with?

Example B: basket with several products

YOU	I want 2 cartons of eggs and 1 coffee. What is the total?
TOOL CALL	<code>check_stock(product="eggs")</code>
TOOL RESULT	8 units in stock
TOOL CALL	<code>check_stock(product="coffee")</code>
TOOL RESULT	5 units in stock
TOOL CALL	<code>calculate_total(items=[{"product": "eggs", "quantity": 2}, {"product": "coffee", "quantity": 1}])</code>
TOOL RESULT	49.00 AED
AGENT	Both are available. 2 eggs at 12.00 AED plus 1 coffee at 25.00 AED gives a total of 49.00 AED.
NOTE	The model may call the tools one after another or several in the same step. Check the trace to see which it chose.

Example C: not enough stock

YOU	Can I buy 20 bottles of milk?
TOOL CALL	<code>check_stock(product="milk")</code>
TOOL RESULT	12 units in stock
AGENT	Sorry, we only have 12 bottles of milk right now, so I cannot confirm 20. Would you like 12 instead?

Example D: out of stock

YOU	How much is a loaf of bread?
TOOL CALL	<code>get_price(product="bread")</code>
TOOL RESULT	4.50 AED
TOOL CALL	<code>check_stock(product="bread")</code>
TOOL RESULT	0 units in stock
AGENT	Bread costs 4.50 AED, but unfortunately it is out of stock at the moment. Can I help you with something else?

Example E: product that does not exist

YOU	Do you sell bananas?
TOOL CALL	<code>check_stock(product="bananas")</code>
TOOL RESULT	Product not found
AGENT	I am sorry, we do not carry bananas. We do have milk, bread, eggs, apples and coffee. Would any of these work?

Example F: off-topic question

YOU	What is the weather like today?
AGENT	I am only able to help with questions about our store, such as prices and availability. Is there anything you would like to know about our products?
NOTE	No tool is called. If the trace shows one, your system prompt needs to be stricter.

Example G: memory across turns (one session)

YOU	Do you have apples?
TOOL CALL	<code>check_stock(product="apples")</code>
TOOL RESULT	30 units in stock
AGENT	Yes, we have apples in stock.
YOU	How much for 5 of them?
TOOL CALL	<code>get_price(product="apples")</code>
TOOL RESULT	3.00 AED
TOOL CALL	<code>calculate_total(items=[{"product": "apples", "quantity": 5}])</code>
TOOL RESULT	15.00 AED
AGENT	5 apples come to 15.00 AED.
YOU	Add 2 coffees to that and give me the new total.
TOOL CALL	<code>check_stock(product="coffee")</code>
TOOL RESULT	5 units in stock
TOOL CALL	<code>calculate_total(items=[{"product": "apples", "quantity": 5}, {"product": "coffee", "quantity": 2}])</code>
TOOL RESULT	65.00 AED
AGENT	Done! 5 apples and 2 coffees come to 65.00 AED in total.
NOTE	In the last two turns the customer never repeats the product or the quantity. The agent can only answer because the earlier messages were sent with each turn. Without memory it should ask what "them" refers to.

Example H: resuming a session (SQLite only)

NOTE	The script was closed and started again, and the customer typed the same session id from Example G.
YOU	What was my last total?
AGENT	Your last total was 65.00 AED, for 5 apples and 2 coffees. Would you like to change anything?
NOTE	No tool call is needed, because the number comes from an earlier tool result stored in the history.

Example I: how the terminal should look while streaming

```
you> Do you have milk, and how much would 3 cost?
[tool] check_stock(product="milk") ... done
[tool] get_price(product="milk") ... done
[tool] calculate_total(items=[{"product": "milk", "quantity": 3}]) ... done
agent> Yes, we have milk in stock! It costs 7.00 AED each, so 3 bottles come to 21.00 AED.
```

The tool lines appear the moment each tool runs, and the agent text then grows word by word instead of appearing in one block.

10. Checking the traces on the OpenAI platform

After running the examples, open the traces dashboard and go through the points below. This is the most valuable part of the exercise.

What to check	What it tells you
Workflow name and shared session identifier	All turns of one chat appear together when you filter by the session id. Confirm that a new session gets a different identifier.
Order of spans inside a turn	The sequence of model calls and tool calls. Compare Example A (three tools) with Example F (no tools).
Arguments generated for the tools	In Example G, open the turn "How much for 5 of them?" and confirm the model filled in apples on its own. This proves the memory reached the model.
Input of the model call in a later turn	Open the model call of the third turn in Example G and confirm that the earlier messages are part of its input.
Duration of each span	Where the time goes: model thinking, tool execution, or the extra round trip after the last tool result.
A streamed turn versus the final text	Confirm that the streamed answer you saw in the terminal matches the output recorded in the trace.

11. Experiments

Try at least two of these once the main exercise works.

Experiment	What to observe
Make the system prompt vague, then very strict	Does the agent still call tools for every number? Does it start guessing prices?
Remove the docstring from one tool	Does the model still choose the right tool? How do the arguments change?
Ask with typos ("mlik") or in another language	How does the agent handle a product name that does not match exactly?
Send only the last 4 messages as memory, then ask a follow-up that depends on the first turn	When does the agent lose the context? How does the trace input change?
Disable memory completely and repeat Example G	Does the agent ask for clarification or does it guess?
Add a fourth tool, <code>apply_discount(code, total)</code> , where the code <code>SAVE10</code> gives 10% off	Does the agent use it on its own after "I have the code <code>SAVE10</code> "? On the 65.00 AED basket the expected result is 58.50 AED.

12. Done when

Examples A to G (and H if you chose SQLite) behave as described, the answers stream in the terminal, every session is visible in the traces dashboard under your workflow name with its turns grouped, and you can explain from the trace why each tool was called and how memory influenced the later turns.

PHASE 2: The Corner Store Team

Multi-agent orchestration: parallel agents, agents as tools, handoffs and owner notifications

13. What Phase 2 is

Sections 1 to 12 are **Phase 1**: one agent with tools, memory, streaming and tracing. **Phase 2** does not start a new project. You keep your Phase 1 code and add four steps on top of it, in order. Each step introduces one way of making agents work together, so you feel the trade-offs yourself instead of only reading about them.

The scenario: the store is doing well and the owner wants more than a chat assistant. They want to be **notified** when something important happens (an order, low stock), to send a **weekly promotion** that is well written, and to stop depending on one big agent that does everything.

Starting point: what you already have from Phase 1

Piece	Phase 1 state
Agent and tools	One Corner Store Assistant with the six business rules, and the tools <code>check_stock</code> , <code>get_price</code> , <code>calculate_total</code> and <code>apply_discount</code> .
Chat features	Conversation memory, streamed output with <code>[tool]</code> lines, tracing with a shared session id.
Store data	The inventory dictionary and the discount codes.

Rule for every step: the Phase 1 chat must keep working. After each step, re-run Examples A to G before moving on.

14. Phase 2 roadmap

Do the steps in this order. Each one builds on the previous one.

Step	What you add on top	New or changed	Started by	Pattern practiced
2.1 Owner notifications	A way for the system to reach the owner.	New: <code>send_message</code> , <code>notify_owner</code> tool.	The other steps. You test it with a small script.	Fallback strategy, function tool
2.2 Weekly promo	A separate script that writes the owner a promo.	New: three writers, a picker, a sender. Chat unchanged.	You, running the script by hand.	By code: parallel agents, forced tool use
2.3 Specialists	The chat agent becomes a manager of specialist agents. The promo gets an LLM manager too.	Changed: the chat agent. New: two specialists, Promo Manager.	2.3a: the chat. 2.3b: you, running a second script.	Agents as tools, <code>draw_graph</code>
2.4 Checkout	Customers can place an order, and the owner is told.	New: <code>place_order</code> , Checkout Agent. Changed: the inventory.	A customer message in the chat.	Handoffs

Concepts from the orchestration lesson (Week 2, Day 2)

Concept	What it means
Orchestrating by code	Your Python decides the order of the steps. Predictable and easy to debug, but the flow is fixed.
Parallel agents	Several agents run at the same time with <code>asyncio.gather</code> and <code>Runner.run</code> , inside one trace.
Forced tool use	<code>ModelSettings(tool_choice="required")</code> makes an agent call a tool instead of just replying.
Agent as a tool	<code>agent.as_tool(...)</code> : a manager calls another agent like a function and control comes back (A to B to A). Good for planner agents.
Handoff	<code>handoffs=[...]</code> : an agent passes control to another one, which answers from then on (A to B).
Traces and graphs	<code>trace(...)</code> groups a whole workflow; <code>draw_graph(agent)</code> shows who can call whom.

15. Step 2.1: Owner notifications

Practices: fallback strategy, turning a plain function into a tool, inspecting the generated tool schema.

Builds on: nothing from Phase 2 yet. This is the foundation the next steps call.

Write one function, `send_message(subject, text_body, html_body)`, that tells the owner something. Everything else in this part calls it, so no other code should know which channel is used.

Requirement	Detail
Fallback channel	Always works with no setup: print the message to the terminal and append it to <code>owner_notifications.log</code> with a timestamp.
Real channel (pick one)	Email through SMTP (<code>EMAIL_ADDRESS</code> , <code>EMAIL_SMTP_SERVER</code> , <code>EMAIL_APP_PASSWORD</code> ; for Gmail you need 2-step verification and an app password) or a Pushover push (<code>PUSHOVER_USER</code> , <code>PUSHOVER_TOKEN</code>). Send to yourself only. Email may land in spam.
Automatic choice	If the real channel's variables are missing, or sending fails, use the fallback. Never crash the chat loop.
Secrets	Read from <code>.env</code> , never written in the code. Add the new names to <code>.env.example</code> .
As a tool	Wrap it with <code>@function_tool</code> as <code>notify_owner(subject, text_body, html_body)</code> with a full docstring.

Pushover has no HTML, so with that channel `html_body` is accepted but unused. Nothing starts this step on its own: write a small script, for example `src/notify_test.py`, that calls `send_message` once (Example J). Then print `notify_owner.params_json_schema` and compare it with your docstring. The SDK built that schema for you, and it is what the model actually sees.

Checkpoint: Example J works: a test script that calls `send_message` delivers through the real channel, and with the variables removed it uses the fallback without an error.

16. Step 2.2: The weekly promo, orchestrated by code

Practices: three agents in parallel, a picker agent, forced tool use, one trace for the whole workflow.

Builds on: Step 2.1 (notify_owner) and the Phase 1 store data.

What starts it

The promo is **not part of the chat**, and no customer message or model starts it. It is a separate script, for example `src/promo.py`, and the trigger is **you running it** in the terminal (`uv run src/promo.py`). The word "weekly" is only the scenario: in a real shop a scheduler such as cron would run the same script every Monday. For this exercise, you are the scheduler.

What it does when it runs

The script is a fixed pipeline. **Your Python code** decides the order of the steps, and no model chooses what happens next. Models are only used inside steps 1 to 3.

Step	Who does it	What it must do
0. Brief	Your code, no LLM	Read the store data and build a short brief: the product with the most stock is the featured product, plus the active discount code. Example: "Feature apples (30 in stock, 3.00 AED) and the code SAVE10".
1. Write	Three writer agents, at the same time	All three receive the same brief and the same store introduction, but each has a different style: warm neighbourhood shop, witty, and short and direct. Run them in parallel with <code>asyncio.gather</code> .
2. Pick	A picker agent	Receives the three drafts and replies with the best one only, with no explanation.
3. Send	A sender agent	Receives the chosen draft and has the <code>notify_owner</code> tool with <code>tool_choice="required"</code> , so it must send it. The owner gets the promo as a push notification, as a preview.

Put the whole script inside one with `trace("Promo workflow"): block`, so the trace shows all steps together. Stream the picker's answer to the terminal with `Runner.run_streamed`, reusing your Phase 1 streaming code.

Questions to answer in your notes: Why does the sender need forced tool use but the picker does not? How much faster is the parallel step than running the writers one by one? (Compare the span durations.) What would break if the picker added an explanation before the email?

Checkpoint: Running `uv run src/promo.py` ends with a push on your phone (Example K), and the trace shows one trace, three overlapping writers and exactly one `notify_owner` call.

17. Step 2.3: Specialists, orchestrated by an LLM (agents as tools)

Practices: `as_tool`, *manager (planner) agents*, `draw_graph`, *comparing code and LLM orchestration*.

Builds on: Phase 1 tools and system prompt (2.3a), and the writers and `notify_owner` from Steps 2.1 and 2.2 (2.3b).

2.3a Split the assistant into specialists

Agent	Tools it owns	Role
Inventory Specialist	<code>check_stock</code>	Answers only about availability.
Pricing Specialist	<code>get_price</code> , <code>calculate_total</code> , <code>apply_discount</code>	Answers only about prices, totals and discounts.
Shopkeeper (manager)	the two specialists, as tools	Talks to the customer, follows the six business rules from Phase 1, and owns no function tools itself.

Create the specialist tools with `agent.as_tool(tool_name=..., tool_description=...)`. The description is what the Shopkeeper reads to decide which one to call, so write it carefully. Replace the Phase 1 agent in your chat loop with the Shopkeeper, so customers talk to it. Run `draw_graph` on the Shopkeeper and keep the picture. Examples A to G from Phase 1 must still behave the same, including memory.

2.3b A promo manager that plans for itself

Redo Step 2.2 with an LLM in charge. Create a **Promo Manager** whose tools are the three writers (as tools) and `notify_owner`. Its prompt must list numbered steps: get a draft from each writer, choose the best, send exactly one notification. Do not write the pipeline in code this time. Like Step 2.2 it is a separate script you run by hand (for example `src/promo_manager.py`), and you give the manager the same brief that step 0 of Step 2.2 builds, so reuse that code.

Compare with Step 2.2: run each version five times. Count how many runs sent exactly one notification, and compare the number of spans, the total time and the token usage in the traces. Write down which version you would trust in production and why.

Checkpoint: Examples A to G still pass through the specialists, Example L shows the nested calls, and you have your five-run comparison of 2.2 versus 2.3b.

18. Step 2.4: The checkout, using a handoff

Practices: *handoffs*, *control passing across (A to B)*, *what happens to the next chat turn*, *notifications from an agent*.

Builds on: the Shopkeeper from Step 2.3 and `notify_owner` from Step 2.1.

Customers can now place an order. Add a `place_order(items)` tool that checks the stock, lowers the stock in the in-memory inventory, and returns an order number starting at 1001 together with the units left of each ordered product. The inventory lives in memory, so it resets when you restart the chat. Then create a **Checkout Agent**:

Item	Detail
Tools	calculate_total, apply_discount, place_order, notify_owner
Instructions	Confirm the basket, apply the discount code if the customer gave one, place the order, then notify the owner with the order summary. If any product is down to 5 units or fewer after the order, the notification must also say LOW STOCK with the product and what is left (it reads this from the place_order result).
How it is reached	The Shopkeeper has handoffs=[checkout_agent] and hands off when the customer clearly wants to buy ("place the order", "I'll take it"), not when they only ask for a price.

The important detail: a handoff passes control *across*, so the Checkout Agent writes the final answer. In a chat loop that raises a question the lesson does not answer for you: on the *next* customer message, which agent should run? Look at `result.last_agent` and decide. Test both options (always restart with the Shopkeeper, or continue with the last agent) and write down what changes. If the Checkout Agent sometimes skips the notification, try forcing tool use as in Step 2.2.

Checkpoint: Example M works: the order is placed, the stock is lower on the next question, and the owner is notified with the low stock warning.

19. Example behaviors for Phase 2

As in Phase 1, the wording of the agents will vary. The tool usage and the numbers should match. Examples use the original five-product data.

Example J: owner notification (Pushover, with fallback)

```
$ uv run src/notify_test.py
[notify] sent with pushover
(without the Pushover variables:)
[notify] using fallback
[notify] 2026-10-06 14:02:11 | Test | If you can read this, notifications work.
```

Example K: the promo workflow (orchestrated by code)

```
$ uv run src/promo.py
[brief] featured: apples (30 in stock, 3.00 AED) | code: SAVE10
[writers] 3 drafts ready in 3.1 s (parallel)
[picker] best draft chosen
agent> Fresh apples at 3.00 AED a kilo this week! Show the code SAVE10 at the till for 10% off.
[sender] notify_owner(subject="Weekly promo: apples", ...) ... done
```

NOTE Open the trace "Promo workflow": the three writers must overlap in time, and the sender's span must contain one `notify_owner` call.

Example L: the Shopkeeper using its specialists (agents as tools)

YOU	Do you have coffee, and how much for 2 with the code SAVE10?
TOOL CALL	inventory_specialist(input="Is coffee in stock?")
TOOL RESULT	5 units in stock
TOOL CALL	pricing_specialist(input="Total for 2 coffee, then apply SAVE10")
TOOL RESULT	50.00 AED, 45.00 AED with SAVE10
AGENT	Yes, we have coffee! 2 come to 50.00 AED, and with SAVE10 you pay 45.00 AED.
NOTE	In the trace, the specialists' own tool calls (check_stock, calculate_total, apply_discount) appear nested inside the Shopkeeper's calls.

Example M: placing an order (handoff to checkout)

YOU	I'll take 2 eggs and 1 coffee. I have the code SAVE10, please place the order.
HANDOFF	Shopkeeper to Checkout Agent
TOOL CALL	calculate_total(items=[{"product": "eggs", "quantity": 2}, {"product": "coffee", "quantity": 1}])
TOOL RESULT	49.00 AED
TOOL CALL	apply_discount(code="SAVE10", total=49.00)
TOOL RESULT	44.10 AED
TOOL CALL	place_order(items=[{"product": "eggs", "quantity": 2}, {"product": "coffee", "quantity": 1}])
TOOL RESULT	Order 1001 placed. Left in stock: eggs 6, coffee 4
TOOL CALL	notify_owner(subject="New order 1001", text_body="2 eggs, 1 coffee, 44.10 AED. LOW STOCK: coffee (4 left)", ...)
TOOL RESULT	Notification sent
AGENT	Order 1001 is placed: 2 eggs and 1 coffee for 44.10 AED with SAVE10. Thank you!
YOU	Do you still have coffee?
TOOL CALL	check_stock(product="coffee")
TOOL RESULT	4 units in stock
AGENT	Yes, we have 4 left.
NOTE	The last answer proves the order really lowered the stock (5 before, 4 now). Which agent answered the second question depends on your choice about result.last_agent.

20. Checking the traces for Phase 2

What to check	What it tells you
The "Promo workflow" trace (Step 2.2)	One trace holds the writers, picker and sender. The three writer spans overlap in time because they ran in parallel.
Writer spans versus manager spans	In Step 2.2 your code made the calls. In Step 2.3b the manager made them: compare how many spans there are and who decided each step.
Nested agent spans (Step 2.3a)	A specialist's tool calls sit inside the span of the Shopkeeper's tool call. Control returned to the Shopkeeper, which is the A to B to A pattern.
The handoff span (Step 2.4)	After the handoff, the rest of the turn runs under the Checkout Agent. Control did not come back.
The draw_graph picture	Compare the Step 2.3a graph (tools) with the Step 2.4 graph (handoff). The arrows are different.
Arguments of notify_owner	Did the model write a useful subject and a text body that matches the HTML body?

21. Experiments

Try at least three of these once the four steps work.

Experiment	What to observe
Remove <code>tool_choice="required"</code> from the sender in Step 2.2 and run it 10 times	How often does the sender reply with text and send nothing?
Replace the picker agent with a rule in code (for example the shortest draft)	Is the result worse? What did you gain in speed, cost and predictability?
Give the three writers different models (a nano and a mini model)	Is the quality difference visible? What does it do to cost and time in the trace?
Make the Promo Manager's steps vague ("get good promos and send one")	Does it send zero, one or several notifications? Does it skip a writer?
Remove the <code>tool_description</code> from one <code>as_tool</code> specialist	Does the Shopkeeper still pick the right specialist? What does it write as input?
Build the checkout as an agent tool instead of a handoff	Who writes the final reply now? What changes in the trace and in the next turn?
Ask the Shopkeeper for 20 milk and to place the order	Does the stock rule from Phase 1 still hold after the handoff, or does the Checkout Agent ignore it?

22. Stretch challenges

Challenge	What to do
Refine the promo (by code)	In your Step 2.2 script, after the picker, add a loop: a critic agent reviews the chosen promo and answers APPROVED or gives feedback, and a rewriter applies it. Stop at APPROVED or after 3 rounds, whichever comes first. Why does the loop need a maximum?
Refine the promo (by an LLM)	Give the Promo Manager the critic and rewriter as tools and let it decide when to stop. Compare with the loop in code.
Hard: two-way email	Switch to a professional email provider such as SendGrid or Resend, receive the owner's reply (for example "restock coffee") and have an agent answer and keep the conversation going.

23. Reflection

Answer in a short notes file. Use your traces as evidence, not opinions.

1. For each of the three patterns (code, agent as tool, handoff), name one thing in this store that it fits best and one it fits badly.
2. Which pattern was the least reliable for you, and which prompt change made it better?
3. Where in your own work could an agent team like this replace a manual process?

24. Done when (Phase 2)

Step 2.1 delivers a notification through the real channel and falls back by itself when it is unavailable. Step 2.2 runs as one trace with three overlapping writers and exactly one notification. Step 2.3 gives the same answers as Phase 1 through specialists, and the 2.3b manager can be compared with Step 2.2 using your five runs. Step 2.4 places an order, lowers the stock and notifies the owner, including a low stock warning. You can explain from the traces and graphs why each pattern behaved the way it did.